

Guia de Banco de Dados Relacional

Modelagem, Cardinalidade e SQL — material de apoio para o desafio de organização financeira

Resumo dos conceitos de banco de dados relacional (entidades, cardinalidade e SQL), organizado como guia de consulta rápida. Os exemplos usam sintaxe compatível com **PostgreSQL**, com uma seção dedicada às diferenças em relação ao MySQL/MariaDB. Ao final há uma proposta de modelagem aplicada a um sistema de organização financeira, como ponto de partida para o desafio.

1. Banco de Dados Relacional e SGBD

Um **SGBD** (Sistema de Gerenciamento de Banco de Dados) é o software responsável por armazenar, organizar e permitir a manipulação de dados. Em um SGBD **relacional**, os dados são organizados em **tabelas** compostas por linhas (registros) e colunas (atributos), e as tabelas se relacionam entre si por meio de chaves.

Exemplos de SGBDs relacionais: MySQL/MariaDB, PostgreSQL, SQL Server, Oracle. Todos usam a linguagem **SQL** (Structured Query Language) como base, com pequenas variações de sintaxe entre eles — ver seção 7.

A linguagem SQL se divide em três grupos principais de comandos:

- **DDL** (Data Definition Language) — define a estrutura: **CREATE**, **ALTER**, **DROP**.
- **DML** (Data Manipulation Language) — manipula os dados: **INSERT**, **UPDATE**, **DELETE**.
- **DQL** (Data Query Language) — consulta os dados: **SELECT**.

2. Modelagem Conceitual (MER)

Antes de criar tabelas, é preciso entender como as informações se relacionam. Isso é feito através do **Modelo Entidade-Relacionamento (MER)**, que representa:

- **Entidades** — algo do mundo real que queremos representar (ex.: Cliente, Produto, Conta, Transação).
- **Atributos** — características de uma entidade (ex.: Cliente possui nome, e-mail).
- **Relacionamentos** — associações entre entidades (ex.: Cliente *realiza* Pedido).
- **Cardinalidade** — quantas ocorrências de uma entidade podem se relacionar com outra.

Um relacionamento sozinho não diz quantas ocorrências existem de cada lado — quem responde essa pergunta é a **cardinalidade**, tema da próxima seção.

3. Cardinalidade

Cardinalidade define a quantidade de relações possíveis entre duas entidades. Existem três tipos básicos:

3.1 Um para Um (1:1)

Cada registro de uma entidade se relaciona com **no máximo um** registro da outra, e vice-versa.

Exemplo	Regra
Pessoa ↔ CPF	cada pessoa tem um CPF; cada CPF pertence a uma pessoa
Funcionário ↔ Armário	cada funcionário tem um armário; cada armário pertence a um funcionário

3.2 Um para Muitos (1:N)

O relacionamento mais comum. Um registro de uma entidade pode se relacionar com **vários** registros da outra, mas cada registro do outro lado se relaciona com apenas um.

Exemplo	Regra
Cliente → Pedido	um cliente faz vários pedidos; cada pedido pertence a um cliente
Conta → Transação	uma conta possui várias transações; cada transação pertence a uma conta

Representação: Cliente (1) — (N) Pedido — a chave estrangeira fica sempre do lado "N".

3.3 Muitos para Muitos (N:N)

Ambos os lados podem se relacionar com vários registros do outro lado.

Exemplo	Regra
Aluno ↔ Disciplina	um aluno cursa várias disciplinas; uma disciplina tem vários alunos
Produto ↔ Pedido	um pedido contém vários produtos; um produto aparece em vários pedidos

Bancos relacionais não implementam N:N diretamente. A solução é criar uma **tabela associativa** (ou "tabela de junção") que quebra o N:N em dois relacionamentos 1:N:

```
Aluno (1) --- (N) Matricula (N) --- (1) Disciplina
```

No exemplo financeiro do desafio, isso aparece em algo como Transação ↔ Categoria (se uma transação puder ter várias categorias) ou Orçamento ↔ Categoria.

3.4 Cardinalidade mínima e máxima

Além do tipo (1:1, 1:N, N:N), a cardinalidade também expressa um mínimo e um máximo, no formato (mínimo, máximo):

Notação	Significado	Exemplo
(0, 1)	opcional, no máximo um	Pessoa (0,1) CNH — nem toda pessoa tem CNH
(1, 1)	obrigatório, exatamente um	CPF (1,1) Pessoa — todo CPF pertence a exatamente uma pessoa
(0, N)	opcional, pode ter vários	Cliente (0,N) Pedido — pode nunca ter comprado, ou ter comprado várias vezes
(1, N)	obrigatório, pelo menos um	Pedido (1,N) Item — todo pedido tem pelo menos um item

4. Definindo Tabelas (DDL)

Criando um banco de dados e conectando a ele:

```
CREATE DATABASE financas;
```

Criando uma tabela com chave primária:

```
CREATE TABLE cliente (
  id_cliente SERIAL PRIMARY KEY,
  nome       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) UNIQUE
);
```

Criando uma tabela relacionada, com **chave estrangeira** (FOREIGN KEY) — implementação de um relacionamento 1:N:

```
CREATE TABLE conta (
  id_conta SERIAL PRIMARY KEY,
  id_cliente INT NOT NULL,
  nome_conta VARCHAR(50),
  saldo      NUMERIC(12,2) DEFAULT 0,
  FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);
```

Outros comandos DDL úteis:

```
ALTER TABLE conta ADD COLUMN ativa BOOLEAN DEFAULT TRUE;
DROP TABLE conta;
```

5. Manipulando Dados (DML)

5.1 Inserir

```
INSERT INTO cliente (nome, email)
VALUES ('Marcos', 'marcos@email.com');
```

5.2 Atualizar

```
UPDATE conta
SET saldo = saldo - 150.00
WHERE id_conta = 1;
```

5.3 Excluir

```
DELETE FROM conta WHERE id_conta = 1;
```

6. Consultando Dados (DQL)

6.1 SELECT básico

```
SELECT nome FROM cliente;
SELECT * FROM cliente WHERE id_cliente = 3;
SELECT nome FROM cliente ORDER BY nome;
```

6.2 Operadores da cláusula WHERE

Operador	Significado
=	igual
<> ou !=	diferente
>, >=, <, <=	maior, maior ou igual, menor, menor ou igual
BETWEEN	valor dentro de um intervalo
IN	valor pertence a uma lista
LIKE	comparação de texto com máscaras (% = vários caracteres, _ = um caractere)
IS NULL	seleciona valores nulos

Exemplos:

```
SELECT * FROM transacao
WHERE valor BETWEEN 100 AND 500
ORDER BY valor;

SELECT * FROM cliente
WHERE email LIKE '%@gmail.com';

SELECT * FROM transacao
WHERE categoria IN ('alimentacao', 'transporte');

SELECT * FROM conta
WHERE saldo IS NULL;
```

6.3 JOIN — unindo tabelas relacionadas

Para combinar dados de tabelas ligadas por chave estrangeira:

```
SELECT t.id_transacao, t.valor, t.data_transacao, c.nome_conta
FROM transacao t
JOIN conta c ON t.id_conta = c.id_conta
ORDER BY t.data_transacao;
```

(existe também a forma antiga com vírgula e WHERE no lugar de JOIN/ON — funciona, mas JOIN . . . ON é a forma recomendada por deixar clara a condição de ligação)

6.4 Funções de agregação

Função	Uso
COUNT(*)	conta linhas
SUM(coluna)	soma valores
AVG(coluna)	média
MIN(coluna) / MAX(coluna)	menor / maior valor

Geralmente combinadas com GROUP BY para agregar por categoria — muito útil em relatórios financeiros:

```
SELECT categoria, SUM(valor) AS total_gasto
FROM transacao
WHERE tipo = 'despesa'
GROUP BY categoria
ORDER BY total_gasto DESC;
```